

EngineIQ — Project Defence Q&A Sheet

IoT & ML-Based Predictive Car Engine Health Monitoring System Iddrisu Faadila Wumpini · Lartey-Mensah Emmanuel Tetteh · Group 11 Supervisor: Dr. R.O.M Gyening · Department of Computer Science, KNUST

How to use this: each answer is written to be *spoken*, not read. Aim for 30–60 seconds per answer. Say the first sentence exactly as written — it's the one that sets the frame — then expand naturally.

PART 1 — The questions the panel gave you

Q1. Introduction / State your problem

Answer:

Ghana recorded 2,949 road crash fatalities in 2025 — the highest in 35 years — from 14,743 crashes involving 24,938 vehicles and causing 16,714 injuries. Between January and April 2026 alone, another 1,009 people died on our roads.

Vehicle defects are a documented contributor: research on road traffic crashes in Ghana attributes more than 35% of crashes to some form of vehicle defect, linked to inadequate maintenance and the importation of overaged, used vehicles.

But the sharper problem is economic. Maintenance in Ghana is almost entirely *reactive* — a driver visits a fitter after the car has already stopped. When an engine seizes, the owner loses the vehicle, the income the vehicle generates, and the repair money, all at once. For a taxi, trotro or Bolt driver, the vehicle is not a convenience; it is the household's income-generating asset.

The tools that could catch this early are either reactive by design (OBD-II) or priced beyond reach (commercial fleet platforms). That is the gap this project addresses.

⚠ **Correction to make before you speak:** your report says "Ghana Road Safety Authority (GRSA)". The correct body is the **National Road Safety Authority (NRSA)**. Fix this in the document and never say GRSA out loud.

⚠ **Do not claim engine faults cause most accidents.** NRSA attributes the bulk of crashes to speeding. If you overclaim and an examiner knows the data, you lose credibility in the first two minutes. Vehicle defects are a *contributing factor* — that is defensible, and it is enough.

Q2. Why car engine?

Answer:

Three reasons.

First, **cost**: the engine is the single most expensive component to replace, so early detection has the highest financial payoff of anything we could have monitored.

Second, **predictability**: engines fail *gradually*. Bearing wear, misfire, loose mounts — all of these change the vibration signature long before the ECU trips a fault code. That gradual degradation is exactly what creates a prediction window for a machine learning model to exploit. A component that fails instantaneously would offer no such window.

Third, **local relevance**: in Ghana the vehicle is frequently the household's income-generating asset. Engine down means income zero. The consequence of a missed engine fault here is not inconvenience — it is loss of livelihood.

Q3. Justification — why that field, that method, that hardware?

Answer this as three separate trade-offs. Own the cost of each choice; do not present any of them as free.

Why unsupervised learning, not supervised?

Because labelled fault data from a real Ghanaian engine does not exist and cannot be ethically or affordably produced. Nobody will let us deliberately destroy a bearing to generate training labels. Isolation Forest learns *only* what normal operation looks like — which is the only data we can actually collect. Everything that deviates from that learned baseline is flagged. That constraint drove the method choice, not the other way round.

Why Isolation Forest, not LSTM or an autoencoder?

Malhotra's LSTM encoder-decoder is more powerful for contextual anomalies, but it is computationally heavy and its validation was lab-only. Isolation Forest is distribution-free, trains on a few hundred windows, and runs on a laptop with no GPU and no cloud bill. We traded some sensitivity to contextual faults for the ability to deploy the system at all in a resource-constrained setting. That was a deliberate engineering trade-off, not a limitation of ambition.

Why BLE, not Wi-Fi/MQTT?

This was our supervisor's recommendation and it removed two dependencies at once — the Wi-Fi router and the MQTT broker. A car parked anywhere can now be monitored, with no internet connection needed for data acquisition. The cost of that choice, which we state openly in Chapter 5: roughly 10 metres of range, and BLE's standard single-central-device model means one supervising device per sensor unit at a time.

Why ESP32 / MPU6050 / DS18B20?

Total hardware cost below GHS 400, with the ESP32 itself under GHS 100 on the local market. Affordability is not a nice-to-have in this project — it is the design constraint the whole architecture is built around, formalised as NFR-06.

Q4. What is the gap from other systems?

Answer — one line per system, then the closing line. Deliver this crisply; it is the heart of the defence.

- **OBD-II** is *reactive by design*. It reports a threshold already crossed. It cannot see wear developing, and reading the codes requires a scanner most Ghanaian drivers do not own.
- **Zhang & Ji (2019)** used static frequency thresholds — prone to false positives as engine load varies.
- **Malhotra et al. (2016)** built a powerful LSTM, but lab-only and too heavy for edge deployment.
- **Kanawaday & Sane (2017)** proved Isolation Forest works for predictive maintenance — on industrial-grade hardware, financially impractical for an individual vehicle owner.
- **Samsara / Geotab** work perfectly well — at \$25–70 per vehicle per month, in foreign currency, indefinitely.

Closing line: Each existing system solves one axis. None solve cost *and* prediction *and* accessibility together. The gap is the **intersection**, and that intersection is where this project sits.

Q5. If systems already exist, why should anyone pay attention to yours?

Answer:

Because the existing systems are not merely expensive — they are structurally unavailable to this market. A \$25–70 monthly foreign-currency subscription is not a price an individual Ghanaian driver negotiates down; it is a business model built for corporate fleets, and it will never reach the informal transport sector.

Our contribution is threefold:

1. **A price point that changes who can participate** — under GHS 400 in one-time hardware, versus a recurring foreign-currency subscription with no end.

- 2. **Prediction rather than reaction** — we capture active physical telemetry (vibration and surface temperature) to find developing mechanical wear *before* the ECU ever logs a code.
- 3. **A dynamic baseline rather than fixed thresholds** — the model learns each engine's own normal signature instead of applying rigid limits that misfire under changing load and road conditions.

And it is built entirely on open-source tools, so the architecture can be replicated, scaled or commercialised by other developers without licensing cost. That reproducibility is part of the contribution.

Q6. Your citations shouldn't be more than 2 years old

What to do — tonight, before you print:

Your core academic sources are 2008, 2015, 2016, 2017, 2019 and 2021. Add these four 2025 references to Chapter 2, and if you can, add two of them as extra rows in **Table 2.1**. That table is what the panel stares at — two 2025 entries there answers the recency question before it is even asked.

Reference	Why it matters to you
Shah, R., Mittal, V., & Lotwin, M. (2025). Recent Advances in Vibration Analysis for Predictive Maintenance of Modern Automotive Powertrains. <i>Vibration</i> , 8(4), 68. https://doi.org/10.3390/vibration8040068	Your single most relevant recent paper — automotive, vibration, predictive maintenance. Reviews deep learning on vibration signals and notes compact CNNs achieving real-time motor fault detection on embedded hardware, confirming feasibility for on-vehicle inference.
Kolok, P., Hodoň, M., Ševčík, P., Hotz, L., & Remy, N. (2025). Low-Cost IoT-Based Predictive Maintenance Using Vibration. <i>Sensors</i> , 25(21), 6610.	Same hardware class as yours — ESP32 with MEMS accelerometer, tested under imbalance and wear faults, using RMS and FFT, reaching over ~73% detection accuracy. Cite it deliberately: it positions your work as current <i>and</i> competitive.
Arciniegas, S., et al. (2025). IoT device for detecting abnormal vibrations in motors using TinyML. <i>Discover Internet of Things</i> . https://doi.org/10.1007/s43926-025-00142-4	96.5% accuracy on motor bearing faults with 300 ms latency from collection to alert. A benchmark against your NFR-01 latency target, and a natural bridge to your future-work point on frequency-domain features.
Reis, M. J. C. S. (2025). Lightweight Signal Processing and Edge AI for Real-Time Anomaly Detection in IoT Sensor Networks. <i>Sensors</i> , 25(21), 6629.	Supports your edge-processing architecture argument with current work.

If they ask why Liu et al. (2008) is still cited:

Because that is the paper that *defines* the algorithm. Citing the original source for a method is correct academic practice — you cite the origin for the method and current literature for the state of the field. What needs to be recent is the *review of the field*, and that is what we have updated. Said this way, it stops being a weakness.

PART 2 — Questions they will ask that weren't on your list

Q7. Why does your model show 100% recall in one place and 25% in another?

This is the sharpest thing in your report. Raise it yourself before they find it.

The two numbers measure different things. AI4I 2020 rows are independent tabular samples, not a true time series. So a 50-sample window built from a shuffled simulation stream mixes fault rows and normal rows together, diluting the fault signature the feature extractor depends on. The clean, purpose-built split in Section 4.4.1 measures **the detector** — 100% recall on all 12 held-out fault windows. The cumulative run in Section 4.4.3 measures **the dataset's unsuitability for windowing** — 25% recall.

This does not affect the live BLE path, where consecutive samples are genuine, temporally-ordered vibration data.

And we consider this a finding in its own right: detection metrics for this class of system are highly sensitive to how test windows are constructed. We retained and reported both numbers rather than selecting the flattering one.

Q8. Did you test on a real running engine?

Answer directly. Do not hedge. And volunteer the sensor problem before they find it — the CSV is in the repository and one `value_counts()` exposes it.

No. And the bench baseline we did collect turned out to be unusable, for a reason we can explain precisely.

We logged 7,579 readings over 9.7 minutes from the prototype at rest. On inspection, **every one of those 7,579 rows is identical** — `accX` = 0.0154, `accY` = -0.0745, `accZ` = 1.1526, `temp` = 0.00, with exactly one unique value per column. The 302 feature windows therefore collapse to a single distinct feature vector, which is why all 302 anomaly scores are exactly -0.5000: that is the degenerate value Isolation Forest returns when every point in the set is identical, not a score it learned.

Two independent hardware faults produced that:

1. **The MPU6050 stopped being re-read.** The firmware does call `getMotion6()` every 20 ms, and the values are physically plausible — $Z \approx 1.15$ g is gravity — so the sensor initialised correctly and returned one valid reading. But at 16384 LSB/g the noise floor of a real MPU6050 at rest is on the order of 100 LSB; a live sensor cannot return the same four-decimal value 7,579 times in a row. It answered once and then stopped responding on the I2C bus.
2. **The DS18B20 never returned a valid reading at all.** `temperature` is initialised to 0.0 and only overwritten when the read passes the `DEVICE_DISCONNECTED_C` check. It stayed at 0.00 for the entire run, so that check never passed once — a wiring, pin-assignment or missing 4.7 kΩ pull-up fault.

So the BLE path is a **validated pipeline** — the transport, the encryption, the windowing, the scoring and the logging all demonstrably work end to end — but it is **not a validated detector**, and we do not claim it as one. The AI4I path is where our detection metrics come from.

Our recommendation carries this forward: fix the two sensor faults, then re-collect the baseline from an actual running engine, or from a substitute mechanical source such as a small motor or fan with an induced imbalance fault, before the live detection path is considered validated to the same standard as the AI4I path.

Correct these figures in the document before printing: the report says "7,579 readings over 2.5 minutes". The timestamps span **582 seconds = 9.7 minutes**, an actual delivered rate of **about 13 Hz**, not 50 Hz. The 2.5-minute figure came from dividing 7,579 by an assumed 50 Hz sampling rate. Note also that `inference.py` and `train_bluetooth.py` both pass `fs=50.0`; the real BLE notification rate is roughly 13 Hz, so that constant is wrong too and should be measured rather than assumed. It does not affect any reported number — the data is constant either way — but it matters the moment the baseline is re-collected, and it matters for any future frequency-domain work.

Q9. Fault-class precision is only 0.38 — isn't that poor?

12 fault windows against 385 normal windows. Precision collapses under that class imbalance by construction; it reflects the evaluation set, not the detector.

For an early-warning system, recall is the operative metric. A missed fault costs an engine. A false alarm costs a driver a five-minute check. We optimised for the error we can afford.

Q10. Why contamination = 5%? And why is your FPR 5.2%?

The contamination parameter sets the percentile at which the anomaly threshold is auto-tuned from the model's own training scores — 5% gives a threshold of -0.5155 for the AI4I model and -0.5000 for the BLE model.

Be honest here: 5% was a design choice, not a value derived from engine physics. And an observed false-positive rate of 5.2% falling out of a 5% contamination setting is close to circular — the parameter partly determines the outcome. What we can defend is the *mechanism*: auto-tuning the threshold from the contamination percentile proved more robust than hardcoding a value, and it meant the BLE model received a threshold appropriate to its own score distribution rather than inheriting the AI4I model's. **And see Q16:** the threshold now has out-of-sample support — 5.3% FPR on 76 held-out normal windows against 5.2% in-sample — so the figure is no longer purely self-referential.

Q11. How secure is the Bluetooth link?

BLE communication uses pairing with bonding and link-layer encryption (`ESP_LE_AUTH_REQ_SC_BOND`), and the sensor characteristic is marked `ESP_GATT_PERM_READ_ENCRYPTED` , so the GATT layer itself refuses unencrypted reads and subscriptions rather than relying on the security mode alone.

State the limitation before they do: the sensor node has neither a display nor a keypad, so pairing uses the "Just Works" association model. This protects against passive eavesdropping — the threat that matters for this deployment — but not against an active man-in-the-middle present at the exact moment of pairing. Defending against that requires a device able to display or accept a passkey, which this hardware does not have.

Also note: the 128-bit UUIDs are not access control. They are broadcast in the advertising packet and readable by any scanner. They identify the stream; they do not protect it.

Q12. You claim the system is scalable, but also that only one device can connect. Which is it?

Both, and the distinction matters. Scaling is **per vehicle unit**, not multi-user per unit — multiple ESP32 units can be paired to separate laptops or phones for multi-vehicle deployment without any architectural change, which is what NFR-04 specifies. Each ESP32 advertises with its own `DEVICE_ID` (`EngineIQ_Sensor_ESP32-01`), and every logged record carries that identifier, so units are distinguishable in the database.

The single-central-device constraint is inherited from the BLE standard itself, not from our implementation. Overcoming it for fleet-scale deployment is our recommendation for future work — a BLE mesh or a BLE/Wi-Fi hybrid gateway.

Q13. Why did you log normal windows and not just anomalies?

A deliberate design decision. An engine-health history is only interpretable if it records what normal operation looked like, not only the exceptions. Without the normal record, a technician reviewing an exported CSV sees a list of alarms with no context for what the engine's healthy baseline was. 5,122 records were logged over the testing period, and every scored window carries an `is_anomaly` flag, a `source` field, and its severity classification.

Q14. What would you do differently / what's next?

- Re-collect the BLE baseline from a running mechanical source before the live detection path is used for anything beyond a pipeline demonstration.
 - Build realistic fault *time series* for simulation-mode testing rather than shuffling independent AI4I rows.
 - Add frequency-domain features (FFT peak frequency, spectral energy) to complement the current time-domain statistics — better separability for faults that show up as frequency shifts rather than amplitude changes.
 - BLE mesh or hybrid gateway to lift the single-device constraint.
 - Push and SMS notifications so a driver is alerted without keeping the dashboard open.
 - Make the Butterworth cutoff relative to the sampling rate before re-collecting any BLE data — the current fixed 0.3 Hz constant would filter away the very vibration content a running engine produces.
 - Fix the two sensor faults exposed by the frozen baseline: the MPU6050 dropping off the I2C bus after its first read, and the DS18B20 never returning a valid temperature.
 - Replace the single contiguous holdout with **blocked k-fold** cross-validation, which respects window overlap and would use the 385 windows more efficiently than one 80/20 split.
-

PART 2B — Machine-learning fundamentals

Why this section exists: your supervisor asked about **epochs**. That is a fundamentals probe, and it never arrives alone. Everything below sits in the same neighbourhood — training procedure, validation, and the choices inside the preprocessing pipeline. Q15 and Q16 are the two you must be able to deliver without hesitating.

Q15. How many epochs did you train for? / Where are your epochs?

Answer:

Isolation Forest has no epochs, and that is a property of the algorithm rather than something we omitted.

An epoch is one complete pass of the training data through a model that learns **iteratively** — a neural network starts from random weights, computes a loss, updates those weights by backpropagation, and needs the same data passed through repeatedly before it converges. Isolation Forest is not iterative. It builds 200 independent random trees in a **single pass** and it is then finished. There is no loss function, no gradient and no weight update, so there is nothing for a second pass to change: running it again on the same data with the same seed produces exactly the same forest.

Our equivalent capacity parameter is `n_estimators = 200` — the number of trees — with each tree fitted on a random subsample of 256 windows.

If they meant the signal-processing sense of the word, say this instead: in vibration and biomedical signal analysis an *epoch* is a fixed-length segment of a continuous signal, and *epoching* means cutting a stream into those segments. We do that — `process_dataframe()` slides a **50-sample window with a step of 25**, so 50% overlap. That is our epoching stage.

Ask which sense they mean if it is not obvious. The two answers are completely different and guessing wrong looks worse than asking.

Q16. Did you use a train/test split? Did you cross-validate?

This is the strongest follow-up available to an examiner who asked about epochs. Answer it with the numbers.

Yes, for the normal class we hold out a slice that the model never sees during fitting:

	Windows	Threshold	False-positive rate
Training split	308	-0.5137 (calibrated on this split alone)	5.2% (in-sample)
Holdout split	76	same threshold, applied unchanged	5.3% (out-of-sample)

Fault recall from that same holdout-trained model is **100% on all 12 fault windows**.

Say why the split is contiguous rather than random, because this is the part that shows you understood the problem. Our windows overlap by 25 of their 50 raw rows. A random split would put two windows that share 25 identical raw rows on opposite sides of the split, leaking training data straight into the test set and flattering the result. So we split contiguously — the first 308 windows train, the tail holds out — and discard the one window straddling the seam, so no holdout window shares a single raw row with a training window.

The result that matters: out-of-sample false-positive rate of 5.3% against 5.2% in-sample. Those two numbers agreeing is direct evidence the model is **not overfitting** its baseline — it treats normal operation it has never seen essentially the same as normal operation it trained on.

On cross-validation, be honest: we did not run k-fold. With overlapping windows, honest k-fold requires blocked or grouped folds rather than the standard random ones, and with 385 windows the single contiguous holdout was the more

defensible use of the data. Blocked k-fold is a legitimate improvement and we list it as future work.

One more thing to state plainly: the model and scaler we actually save to disk are refitted on all 385 normal windows afterwards. That is standard practice — validate on a holdout, then refit on everything for deployment — and it is why the figures in Chapter 4 are unchanged by adding this validation step.

Q17. The AI4I dataset has failure labels. Why did you train unsupervised and throw them away?

Answer:

We did not throw them away — we used them for **evaluation only, never for training**, and that was deliberate.

The reason is deployment. The system has to be trainable on a real Ghanaian vehicle, where no labelled fault data exists and none can be affordably or ethically produced. If we had trained a supervised classifier on AI4I's failure labels, we would have built something that cannot be retrained on the actual target — the labels exist in the dataset and nowhere in the field.

Training unsupervised on AI4I keeps the training procedure **identical** to the one that must eventually run on a real engine: learn what normal looks like, flag departures from it. The labels then serve their proper purpose, which is measuring whether that procedure actually works. Using labels to evaluate a model that never saw them is not wasteful; it is the only honest way to validate an unsupervised detector.

Q18. What would overfitting mean for an Isolation Forest, and did you check for it?

Answer:

It cannot be read off a loss curve, because there isn't one. For a one-class anomaly detector, overfitting means the forest has modelled its training baseline so tightly that ordinary, benign variation in normal operation falls outside it and gets flagged as a fault. The symptom is not a poor training score — it is a false-positive rate that is low on training data and much higher on normal data the model has never seen.

That is exactly the comparison in Q16, and it is why we ran it: **5.2% in-sample against 5.3% out-of-sample**. The gap is negligible, so on this dataset the model generalises.

The honest caveat is scope: that tells us the forest generalises across the AI4I normal population. It says nothing about generalising to a different engine, which is a transfer question our data cannot answer.

Q19. Why 200 trees? What happens at 50, or at 500?

Answer:

Trees are the capacity knob — the closest thing this algorithm has to an epoch count. Each tree isolates points on a random 256-window subsample, and the anomaly score is the average isolation depth across all of them. More trees reduce the variance of that average, so scores get more stable and repeatable.

The behaviour is convergence, not improvement without limit: past a few hundred trees the averaged score stops moving and additional trees only cost computation. 200 sits on the flat part of that curve while keeping training to a few seconds on a laptop with no GPU, which is the deployment constraint the whole project is built around.

Worth doing if you have an hour before the defence: plot mean anomaly score against `n_estimators` for 10, 25, 50, 100, 200, 500 trees. A convergence curve is the single best visual answer to "where are your epochs" — it is the Isolation Forest equivalent of a loss curve, and having one turns the weakest question in this section into your strongest slide.

Q20. Explain how Isolation Forest actually decides something is an anomaly.

Have this ready — an examiner who asks about epochs may well ask you to explain the mechanism, and a vague answer here undoes the rest.

It exploits the fact that anomalies are **few and different**, so they are easy to separate from everything else.

Each tree picks a feature at random and a split value at random, over and over, cutting the data until each point sits alone. The measure of interest is the **path length** — how many cuts it took to isolate a given point. A point buried inside a dense cluster of normal operation needs many cuts to separate. A point sitting out on its own is cut off in very few. Averaged over 200 trees, that path length becomes a stable score, which `score_samples()` returns in normalised form where **lower means more anomalous**.

Then the decision: we compare that score against a threshold auto-tuned from the training score distribution — -0.5155 for the AI4I model, -0.5000 for the BLE model — and anything below it is flagged.

The efficiency point is worth one sentence, because it is why the method fits this project: cost grows linearly with sample count, with no distance matrix and no kernel, which is what makes 45-dimensional windows tractable on an ordinary laptop.

Q21. Your Butterworth low-pass cutoff is 0.3 Hz. Justify that for a 50 Hz accelerometer.

Do not defend this. Own it — it is a real defect and a prepared admission costs far less than a fumbled defence.

The cutoff is correct for the AI4I dataset and wrong for the live vibration path, because it was tuned for the first and never re-scaled for the second.

`butterworth_filter()` normalises the cutoff against the Nyquist frequency. At the dataset's `fs = 1.0` Hz, a 0.3 Hz cutoff is 0.6 of Nyquist — a mild, appropriate low-pass. At the BLE path's `fs = 50.0` Hz, the same constant becomes 0.012 of Nyquist, still a **0.3 Hz** real cutoff. Measured against our accelerometer columns, that retains under 0.15% of the signal variance.

That is the wrong filter for the job. Bearing wear, misfire and loose mounts appear in the tens-to-hundreds of hertz range, so a 0.3 Hz low-pass removes the fault signature before the feature extractor ever sees it. The cutoff has to scale with sampling rate; a fixed constant shared between a 1 Hz dataset and a 50 Hz sensor cannot be right for both.

Two things that make this survivable: it changes none of our reported results, because the AI4I path uses the correct effective cutoff and the BLE data was constant regardless (Q8). And the fix is a single parameter made relative to `fs`. It is the first item in our future work, and it has to be corrected **before** the BLE baseline is re-collected, or good sensor data would be filtered away by the pipeline.

Q22. Why MinMaxScaler? What happens when a live reading falls outside the training range?

Answer:

MinMax was chosen because our features are bounded physical quantities and we make no Gaussian assumption about them, which StandardScaler implicitly suits better.

Then concede the cost, because it is the real answer to the second half of the question. MinMax is defined entirely by the training minimum and maximum, so it has two consequences we accept. A single outlier in the training baseline compresses every other value toward the middle of the range. And at inference, a live reading outside the training range maps outside $[0, 1]$ — we do not clip it.

For an anomaly detector that second behaviour is arguably correct rather than a bug: a value beyond anything seen in training *should* land in unfamiliar territory and be scored as anomalous. But it is an untested path, we have not characterised how the forest behaves on out-of-range input, and we do not claim it as a designed feature.

Rapid-fire — one-line answers, know all of them

Question	Answer
Why a 50-sample window and a step of 25?	50 samples is roughly one second of context at the intended rate; the 50% overlap doubles the window count and stops a fault being missed because it straddled a boundary. Concede: 50 was not derived from engine rotational frequency, which is how it should ideally be chosen.
45 features from 385 windows — too many?	Trees split on one feature at a time and need no matrix inversion, so correlated features cost far less than in a distance-based method. Also note <code>range</code> and <code>peak2peak</code> are computed with the identical formula, so it is really 40 distinct features across 5 channels.
Your two models have 45 and 36 features. Comparable?	No, and we never compare their scores. Different sensors, different physical scales, so separate models, separate scalers and separate thresholds by design — that is exactly why each gets its own auto-tuned cutoff.
Why <code>score < threshold</code> rather than <code>model.predict()</code> ?	Three reasons, and this one is a genuine design strength — say so. It decouples the decision from the contamination value baked in at fit time; it lets each model carry its own calibrated cutoff; and it supports graded severity (NORMAL / LOW / MEDIUM / HIGH) instead of a binary flag.
Does <code>random_state = 42</code> change your results?	It fixes the tree randomisation so the run is reproducible. With 200 averaged trees, seed sensitivity should be small — but state plainly that we did not run a seed-sensitivity check.
Why not One-Class SVM or Local Outlier Factor?	Linear time, no kernel to choose, and no distance metric to defend in 45 dimensions. LOF needs a neighbourhood size and SVM needs a kernel plus nu, both tuned on fault data we do not have.
Isn't 385 training windows very few?	Yes, and it is precisely why a tree ensemble rather than a neural network. 385 windows across 45 features would overfit an autoencoder or LSTM badly with no honest validation set left over. The data volume drove the model class.
What is your inference latency?	Tie it to NFR-01 — and measure it before the defence so you can quote a real millisecond figure , rather than saying "fast".

PART 2C — Security, sensor physics, and deployment reality

Why this section exists: Parts 2 and 2B cover the model. This part covers everything around it — the database, the hardware limits, and whether the system is deployable as described. Q23, Q24 and Q25 are checkable from your own repository in under a minute, so assume a thorough examiner will find them. Each one is survivable if you raise it first and unpleasant if you do not.

Q23. If I sign up on your dashboard, can I see someone else's engine data?

As the system currently stands, yes. Say so first, then say what the fix is — this is a security finding, not a limitation, and it is the single most checkable thing in the project.

Row-level security is enabled on `anomaly_logs` and the read policy grants `select` to the `authenticated` role with the condition `using (true)`. That authenticates users without partitioning data between them: any account that confirms an email address can read every row in the table. There is no `user_id` column on the table at all — records are tagged with a `device_id` identifying which physical ESP32 produced them, but nothing records who owns that unit.

The insert side is more permissive still. The policy is `for insert to anon with check (true)`, and the anon key is published in `index.html` because the browser client needs it. Anyone who views the page source can therefore write arbitrary rows into the anomaly log.

The correct framing: we scoped the prototype to a single-owner deployment — one vehicle, one owner, one dashboard — and within that scope the model is coherent. It is not a multi-tenant system and we do not claim it as one. Making it multi-tenant is a schema change rather than an architectural one: add a `user_id` column defaulting to `auth.uid()`, change the read policy to `using (auth.uid() = user_id)`, and restrict inserts to authenticated writers holding a service key rather than to the anon role.

Do not let them redirect you onto the anon key. A public anon key is normal and intended in Supabase — it identifies the project, it is not a secret, and row-level security is what protects the data. The flaw here is the permissiveness of the policy, not the visibility of the key. Saying that clearly demonstrates you understand the security model rather than having merely enabled a checkbox.

Q24. You sample at 50 Hz. What engine frequencies can you actually detect?

Lead with what you did right, because you did do this one right.

The firmware sets the MPU6050's on-chip digital low-pass filter to roughly 21 Hz (`setDLPFMode(MPU6050_DLPF_BW_20)`) before sampling at 50 Hz. That band-limits the signal below the 25 Hz Nyquist frequency, which is correct anti-aliasing practice and a deliberate design decision rather than a default.

Then state the consequence honestly. A 25 Hz ceiling bounds what the system can observe:

Engine speed	First-order frequency	Status
800 rpm (idle)	13.3 Hz	Within range
1,260 rpm	21 Hz	At the anti-alias filter limit
1,500 rpm	25 Hz	At Nyquist
3,000 rpm	50 Hz	Beyond range

So the system is capable of detecting **low-frequency mechanical phenomena near idle** — imbalance, looseness, degraded engine mounts — and is **not** capable of detecting classic rolling-element bearing or gear-mesh faults, which appear at several multiples of shaft speed and excite structural resonances in the kilohertz range. Detecting those needs a sampling rate an order of magnitude higher, which the BLE notification path in its current form cannot carry.

Correct the Chapter 2 framing accordingly. Where the report describes bearing wear changing the vibration signature, that is true of vibration analysis in general but outside the band this hardware samples. State the detectable band explicitly rather than letting the general claim stand.

One further point, and it is the one that actually needs fixing: the ESP32 samples at 50 Hz but only about 13 Hz of those samples reach the laptop. Dropping samples in transport is decimation, and there is no anti-alias filter in front of it, so content between roughly 6.5 Hz and 21 Hz folds back into the retained band. The on-chip DLPF protects the ESP32's own sampling; nothing protects the BLE link. That is the more important of the two frequency problems, because it undermines a filter stage we otherwise implemented correctly.

Q25. Your accelerometer is configured for $\pm 2g$. Isn't engine vibration larger than that?

Answer:

Yes, on a running engine it will be. The firmware sets `MPU6050_ACCEL_FS_2`, a $\pm 2g$ full-scale range, which maximises resolution — the reason it suited a stationary bench prototype — at the cost of clipping anything beyond 2g. Engine-block vibration routinely exceeds that, and impulsive events certainly do.

Explain why this matters specifically to your feature set, because that is what shows depth. Clipping truncates peaks, and kurtosis is precisely a measure of how peaked a distribution is. A clipped signal has *lower* kurtosis than the true signal, so saturation degrades the one feature most likely to reveal an impulsive mechanical fault. The same applies to peak-to-peak and to the maximum. Of the nine features we extract per channel, clipping corrupts several of the most diagnostically useful.

The remedy is a one-line change to $\pm 8g$ or $\pm 16g$, accepting coarser resolution per count. It must be done **before** the baseline is re-collected, because a model trained at one full-scale range cannot be applied to data captured at another.

Q26. Your system needs a laptop running Python in the vehicle. Is that realistic, and is the laptop inside your GHS 400?

This question attacks your central cost claim. Have the boundary clear.

The GHS 400 covers the sensor unit — the ESP32, MPU6050, DS18B20 and enclosure — which is the article that would be sold. It does not include a computer, and it does not need to: the pairing device is a laptop or phone the owner already has, in the same way a fitness tracker assumes a phone rather than shipping one.

Then be honest about the current state. Today the paired device must run a Python inference process, and that is a development-stage arrangement rather than a consumer one. The productisation path is either on-device inference — the model is a serialised Isolation Forest, small enough that this is realistic — or a phone application performing the same scoring. Both are identified in our future work.

Protect the comparison. The GHS 400 versus 25 to 70 US dollars per month contrast is a comparison of *recurring subscription* against *one-off hardware*, and it holds regardless of the host device, because the commercial platforms assume a smartphone too.

Q27. Your key differentiator is that the model learns each engine's own normal. How does an actual driver do that?

Concede this one cleanly — the capability is real but the workflow is not.

Architecturally, per-vehicle baselining is exactly what the system does: `collect_ble_baseline.py` records normal operation, `train_bluetooth.py` fits an Isolation Forest to that specific engine, and the threshold is auto-tuned from that engine's own score distribution. Nothing about that is generic or shared between vehicles, and that is genuinely different from a fixed-threshold system.

Operationally, it currently requires running two Python scripts from a terminal, which no driver will do. The capability is implemented and the user-facing workflow is not. What is needed is a single guided action in the dashboard — a "learn my engine" control that records for a fixed period, confirms the data is valid, trains, and stores the resulting model against that `device_id`. That is a front-end task on top of machinery that already exists, and it is the most valuable single item in our future work because it is what converts the differentiator from an architectural claim into a usable feature.

Q28. How is the sensor unit powered in a vehicle, and what happens when the engine is off?

Have a concrete answer rather than an improvised one. The ESP32 draws power over USB, supplied in-vehicle from a 12 V accessory socket adapter. Be ready to state what happens at engine-off — whether the unit powers down with the accessory circuit or continues advertising — and to acknowledge that continuous operation on a parked vehicle raises a battery-drain question we have not characterised. Power budgeting under real vehicle conditions is not something we measured.

Q29. Where is the unit mounted, and will the components survive there?

Answer, and volunteer the thermal mismatch:

The DS18B20 is specified to +125 °C and is suitable for engine-surface contact. The MPU6050 is rated to approximately +85 °C, and engine-bay temperatures near the block can exceed that. So the two sensors in our own design have different survivable envelopes, and the accelerometer is the limiting component.

The implications we have not addressed are mounting location, thermal derating and vibration-resistant fixing — and mounting matters twice over, because a loose or compliant mount changes the measured vibration signature and would be learned into the baseline as if it were engine behaviour. A rigid, repeatable mounting point is a prerequisite for the model to mean anything across sessions.

Q30. What happens to your cloud backend at scale, or on the free tier?

Logging every scored window rather than only anomalies is a deliberate decision (Q13), but it does mean row count grows continuously — 5,122 records accumulated during testing alone.

Verify the current Supabase free-tier storage ceiling and inactivity-pause policy before quoting figures, because they change. The point to make is that we understand the two distinct risks: a storage ceiling reached through continuous logging, and the availability risk that free-tier projects are paused after a period of inactivity, which is a poor property for a monitoring service. Mitigations are a retention policy that downsamples or expires old normal windows while preserving anomalies, and a paid tier for any real deployment.

Q31. What are the ethical and data-protection considerations?

Expect this at a Ghanaian institution, and note that it is currently absent from the report.

The system stores personal data: registered email addresses, and a continuous behavioural record of when a specific identifiable person operates their vehicle. In Ghana that falls under the **Data Protection Act 2012 (Act 843)**, which requires a lawful basis for processing, purpose limitation, and data-subject rights over the record.

Be able to state four things: what is collected (vibration, temperature, timestamps, account email), where it is stored (Supabase, hosted outside Ghana, which raises a cross-border transfer question), how long it is retained (currently indefinitely, which should become a stated retention policy), and how consent is obtained (currently only implicit account creation, which is not sufficient).

For the prototype itself, data was collected from our own bench hardware, so no third-party personal data was processed during development.

Q32. Do you have automated tests?

Answer directly. No automated test suite. `preprocessor.py` carries a `__main__` smoke test that verifies feature extraction produces the expected window and feature counts on synthetic data, and beyond that verification was manual — checking outputs against known inputs at each pipeline stage.

Unit tests over the feature extractor and the windowing logic are the obvious gap, and they are the highest-value tests to write because every downstream result depends on those two functions being correct.

Q33. Why does your inference engine ignore database errors?

This one looks like a defect and is actually a design decision — make sure it lands that way.

`log_to_supabase()` wraps its insert in an exception handler so that a failure to log never interrupts monitoring. The reasoning is that logging is secondary to detection: if the network drops or the schema is out of date, the correct behaviour for a monitoring system is to carry on scoring windows and alerting locally rather than to terminate because it could not write history. Failures are reported on the console rather than silently discarded.

The limitation we accept is that records generated during an outage are lost rather than queued. Buffering unsent records locally and flushing them on reconnection is the improvement, and it matters more once the system is expected to produce a continuous maintenance history.

PART 3 — Fix in the document before you print

Free points for any examiner who reads carefully:

Issue	Where	Fix
"Ghana Road Safety Authority (GRSA)"	§1.1	Correct body is National Road Safety Authority (NRSA)
"a large percentage" of accidents, no figure	§1.1	Insert a real figure, or soften to "a documented contributing factor"
Feature count contradiction	§2.6 lists 9 statistics (but "range" and "peak-to-peak" are the same thing); §4.3.2 lists 8, yet claims $45 \text{ features} \div 5 \text{ channels} = 9 \text{ per channel}$	Settle on one list of nine and make both sections identical
Figure numbering drift	§4.3.4 calls the Supabase table Figure 4.3; Table 4.4 cites "Figure 4.2 — 5,122 records logged"; §5.3 cites Figure 4.3 for the dashboard while §4.3.5 calls it Figure 4.4	Renumber consistently end to end
Record count mismatch	FR-11 says 60 records; wireframe §3.8.1 says last 50 events	Pick one
Broken cross-references	§4.3.3 cites "Section 2.4" for the auto-tuned threshold; §4.3.2 cites "Section 2.7.5" for feature design — neither section describes those things	Repoint to the correct sections
No bill of materials	GHS 100 / GHS 400 claimed in §1.4, §2.8, NFR-06	Add an itemised cost table as a backup slide — cost is your central justification, so someone will ask
BLE baseline duration is wrong	§4.4.2, Q8	Report says "7,579 readings over 2.5 minutes". Timestamps span 582 s = 9.7 minutes , about 13 Hz actual. The 2.5 min came from dividing by an assumed 50 Hz
BLE baseline is a single frozen reading	§4.4.2	All 7,579 rows are identical and <code>temp</code> is 0.00 throughout — the MPU6050 stopped responding and the DS18B20 never read successfully. State this as a hardware fault, not as "low variance" (see Q8)
<code>fs = 50.0</code> is assumed, not measured	<code>inference.py</code> , <code>train_bluetooth.py</code>	Real BLE notification rate is roughly 13 Hz. Harmless today (constant data), wrong the moment the baseline is re-collected
Butterworth cutoff does not scale with <code>fs</code>	<code>preprocessor.py</code>	<code>cutoff = 0.3</code> gives a sane $0.6 \times \text{Nyquist}$ at <code>fs = 1.0</code> but a 0.3 Hz real cutoff at <code>fs = 50.0</code> , retaining under 0.15% of vibration variance. Make the cutoff relative to <code>fs</code> (see Q21)

Issue	Where	Fix
Add the out-of-sample result	Ch. 4 results	<code>train.py</code> now reports a 308/76 contiguous holdout: 5.3% FPR out-of-sample vs 5.2% in-sample , 100% fault recall. This is your generalisation evidence — put it in the results chapter, not just in the viva answer
No per-user data isolation	Ch. 5 limitations; security discussion	RLS read policy is <code>using (true)</code> and there is no <code>user_id</code> column — any signed-in user reads every row. State the single-owner scope explicitly as a limitation (see Q23)
Detectable frequency band not stated	§2 vibration discussion, Ch. 5	50 Hz sampling with a ~21 Hz anti-alias filter bounds detection to low-frequency imbalance and looseness near idle. Bearing and gear-mesh faults are out of band — say so rather than letting the general "vibration signature" claim stand (see Q24)
Accelerometer range will clip	Ch. 5 limitations	±2g full scale saturates on a running engine, and clipping specifically degrades kurtosis, peak-to-peak and max. Must change to ±8g/±16g before the baseline is re-collected (see Q25)
MPU6050 thermal limit	Ch. 5 limitations	Rated to about +85 °C against the DS18B20's +125 °C; engine-bay temperatures can exceed the accelerometer's envelope. Mounting location and thermal derating are unaddressed (see Q29)
No data-protection section	Add to Ch. 1 or Ch. 5	Emails plus a behavioural record of an identifiable person fall under Ghana's Data Protection Act 2012 (Act 843) . Cover collection, storage location, retention and consent — a Ghanaian panel will expect this (see Q31)

PART 4 — Demo checklist

Your own honesty feature is the trap. If no live row lands within 60 seconds, the dashboard falls back to demonstration data *and says so on screen*, in front of the panel.

- ☐ Pair the ESP32 to the presenting laptop beforehand; confirm a **live**-labelled reading appears. 10 m range, one central device.
 - ☐ **Record a screen capture of a working live session tonight.** If Bluetooth misbehaves in the room, play the video and keep talking.
 - ☐ **Do not demo signup live.** Email delivery lag, the one-hour confirmation expiry, and the GitHub Pages redirect allow-list are three ways to fail publicly. Sign in with an account created in advance.
 - ☐ Demo the **CSV export** instead — it ties directly back to the Chapter 1 mechanic scenario and cannot fail live.
 - ☐ Confirm the Supabase redirect allow-list still contains the full `/Engine_monitor/` path.
-

PART 5 — Suggested timing (15-minute slot)

Minutes	Section
0–2	Problem — Ghana context, reactive OBD-II, GHS 400 vs \$25–70/month (your strongest slide)
2–4	Architecture — the four layers, one diagram
4–6	Hardware and preprocessing pipeline
6–9	ML and results — including the recall gap, on your own terms
9–12	Demo
12–14	Limitations and future work
14–15	Close, then stop

Leaving the honest limitations as the last thing they hear before questions pre-empts half the panel.

Split the delivery so each of you owns whole sections — but both of you must be able to answer on either half. Panels deliberately ask the quiet one about the other's part.